

UNIT – V GRAPH

Basic Graph Concepts:

Graph is a data structure that consists of following two components:

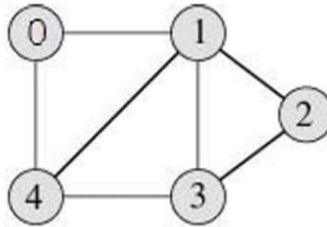
1. A finite set of vertices also called as nodes.
2. A finite set of ordered pair of the form (u, v) called as edge.

The pair is ordered because (u, v) is not same as (v, u) in case of directed graph (di-graph). The pair of form (u, v) indicates that there is an edge from vertex u to vertex v . The edges may contain weight/value/cost.

Graph and its representations:

Graphs are used to represent many real life applications: Graphs are used to represent networks. The networks may include paths in a city or telephone network or circuit network. Graphs are also used in social networks like linkedIn, facebook. For example, in facebook, each person is represented with a vertex(or node). Each node is a structure and contains information like person id, name, gender and locale.

Following is an example undirected graph with 5 vertices.



Following two are the most commonly used representations of graph.

1. Adjacency Matrix
2. Adjacency List

There are other representations also like, Incidence Matrix and Incidence List. The choice of the graph representation is situation specific. It totally depends on the type of operations to be performed and ease of use.

Adjacency Matrix:

Adjacency Matrix is a 2D array of size $V \times V$ where V is the number of vertices in a graph. Let the 2D array be $adj[i][j]$, a slot $adj[i][j] = 1$ indicates that there is an edge from vertex i to vertex j . Adjacency matrix for undirected graph is always symmetric. Adjacency Matrix is also used to represent weighted graphs. If $adj[i][j] = w$, then there is an edge from vertex i to vertex j with weight w .

The adjacency matrix for the above example graph is:

	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

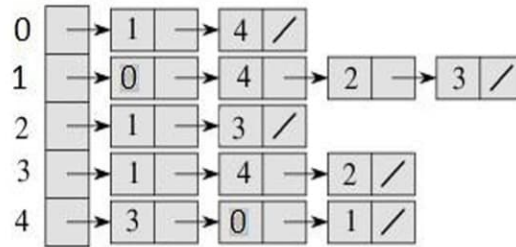
Adjacency Matrix Representation of the above graph

Pros: Representation is easier to implement and follow. Removing an edge takes $O(1)$ time. Queries like whether there is an edge from vertex 'u' to vertex 'v' are efficient and can be done $O(1)$.

Cons: Consumes more space $O(V^2)$. Even if the graph is sparse (contains less number of edges), it consumes the same space. Adding a vertex is $O(V^2)$ time.

Adjacency List:

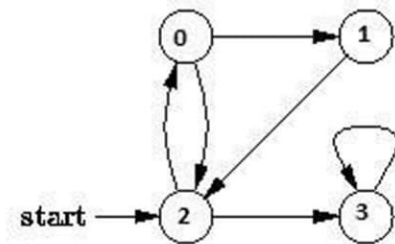
An array of linked lists is used. Size of the array is equal to number of vertices. Let the array be `array[]`. An entry `array[i]` represents the linked list of vertices adjacent to the *i*th vertex. This representation can also be used to represent a weighted graph. The weights of edges can be stored in nodes of linked lists. Following is adjacency list representation of the above graph.



Breadth First Traversal for a Graph

Breadth First Traversal (or Search) for a graph is similar to Breadth First Traversal of a tree. The only catch here is, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a boolean visited array.

For example, in the following graph, we start traversal from vertex 2. When we come to vertex 0, we look for all adjacent vertices of it. 2 is also an adjacent vertex of 0. If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process. Breadth First Traversal of the following graph is 2, 0, 3, 1.



Algorithm: Breadth-First Search Traversal BFS(V, E, s)

for each *u* in $V - \{s\}$

do $color[u] \leftarrow WHITE$ $d[u] \leftarrow infinity$ $\pi[u] \leftarrow NIL$ $color[s] \leftarrow GRAY$

$d[s] \leftarrow 0$ $\pi[s] \leftarrow NIL$ $Q \leftarrow \{s\}$

ENQUEUE(Q, s)

while Q is non-empty

do $u \leftarrow DEQUEUE(Q)$

for each *v* adjacent to *u*

do if $color[v] \leftarrow WHITE$

then $color[v] \leftarrow GRAY$ $d[v] \leftarrow d[u] + 1$

$\pi[v] \leftarrow u$

ENQUEUE(Q, v) DEQUEUE(Q)

$color[u] \leftarrow BLACK$

Applications of Breadth First Traversal

1) **Shortest Path and Minimum Spanning Tree for unweighted graph** In unweighted graph, the shortest path is the path with least number of edges. With Breadth First, we always reach a vertex from given source using minimum number of edges. Also, in case of unweighted graphs, any spanning tree is Minimum Spanning Tree and we can use either Depth or Breadth first traversal for finding a spanning tree.

2) **Peer to Peer Networks.** In Peer to Peer Networks like BitTorrent, Breadth First Search is used to find all neighbor nodes.

3) **Crawlers in Search Engines:** Crawlers build index using Bread First. The idea is to start from source page and follow all links from source and keep doing same. Depth First Traversal can also be used for crawlers, but the advantage with Breadth First Traversal is, depth or levels of built tree can be limited.

4) **Social Networking Websites:** In social networks, we can find people within a given distance 'k' from a person using Breadth First Search till 'k' levels.

5) **GPS Navigation systems:** Breadth First Search is used to find all neighboring locations.

6) **Broadcasting in Network:** In networks, a broadcasted packet follows Breadth First Search to reach all nodes.

7) **In Garbage Collection:** Breadth First Search is used in copying garbage collection using Cheney's algorithm.

8) **Cycle detection in undirected graph:** In undirected graphs, either Breadth First Search or Depth First Search can be used to detect cycle. In directed graph, only depth first search can be used.

9) **Ford-Fulkerson algorithm** In Ford-Fulkerson algorithm, we can either use Breadth First or Depth First Traversal to find the maximum flow. Breadth First Traversal is preferred as it reduces worst case time complexity to $O(VE^2)$.

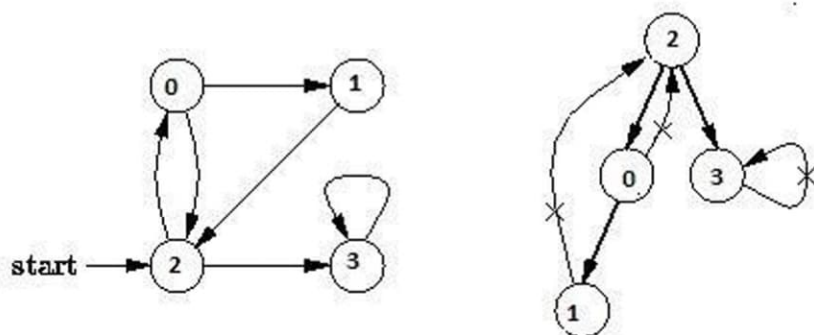
10) **To test if a graph is Bipartite** We can either use Breadth First or Depth First Traversal.

11) **Path Finding** We can either use Breadth First or Depth First Traversal to find if there is a path between two vertices.

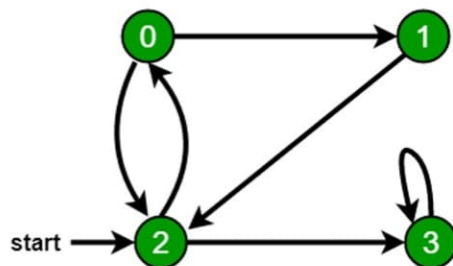
12) **Finding all nodes within one connected component:** We can either use Breadth First or Depth First Traversal to find all nodes reachable from a given node.

Depth First Traversal for a Graph

Depth First Traversal (or Search) for a graph is similar to Depth First Traversal of a tree. The only catch here is, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a boolean visited array. For example, in the following graph, we start traversal from vertex 2. When we come to vertex 0, we look for all adjacent vertices of it. 2 is also an adjacent vertex of 0. If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process. Depth First Traversal of the following graph is 2, 0, 1, 3



Example



When we come to **vertex 0**, we look for all adjacent vertices of it.

- 2 is also an adjacent vertex of 0.
 - If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process.
- There can be multiple BFS traversals for a graph. Different BFS traversals for the above graph :
- 2, 3, 0, 1
2, 0, 3, 1

Pseudocode

- Declare a queue and insert the starting vertex.
- Initialize a visited array and mark the starting vertex as visited.
- Follow the below process till the queue becomes empty:
- Remove the first vertex of the queue.
- Mark that vertex as visited.
- Insert all the unvisited neighbors of the vertex into the queue.

Algorithm Depth-First Search

The DFS forms a depth-first forest comprised of more than one depth-first trees. Each tree is made of edges (u, v) such that u is gray and v is white when edge (u, v) is explored. The following pseudocode for DFS uses a global timestamp time.

DFS (V, E)

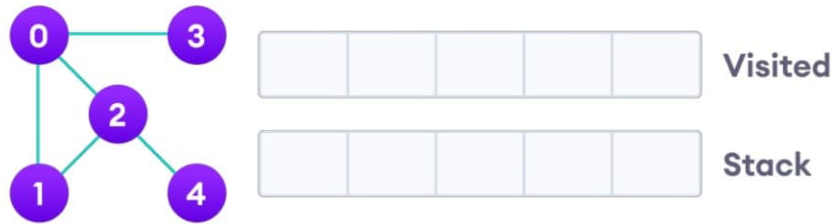
```
for each vertex  $u$  in  $V[G]$ 
do color[ $u$ ]  $\leftarrow$  WHITE  $\pi[u] \leftarrow$  NIL
    time  $\leftarrow$  0
    for each vertex  $u$  in  $V[G]$ 
    do if color[ $u$ ]  $\leftarrow$  WHITE
    then DFS-Visit( $u$ )
```

DFS-Visit(u)

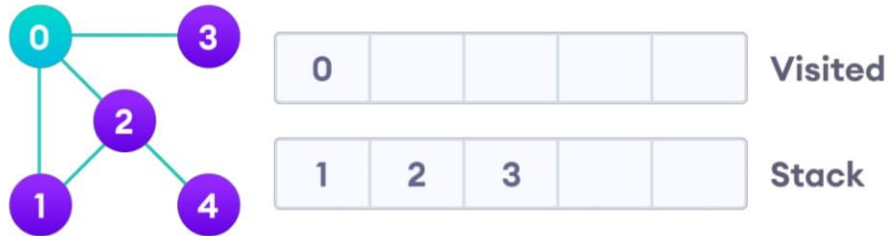
```
color[ $u$ ]  $\leftarrow$  GRAY time  $\leftarrow$  time + 1  $d[u] \leftarrow$  time
for each vertex  $v$  adjacent to  $u$ 
do if color[ $v$ ]  $\leftarrow$  WHITE
then  $\pi[v] \leftarrow u$ 
    DFS-Visit( $v$ ) color[ $u$ ]  $\leftarrow$  BLACK time  $\leftarrow$  time + 1
 $f[u] \leftarrow$  time
```

Pseudocode:

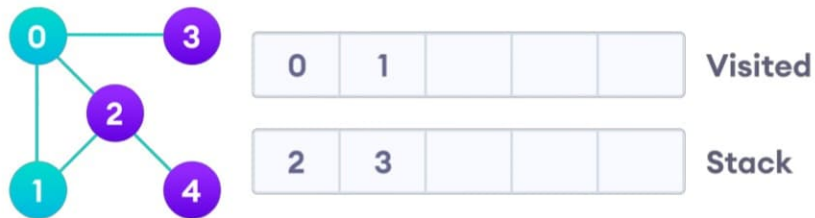
1. Start by putting any one of the graph's vertices on top of a stack.
2. Take the top item of the stack and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
4. Keep repeating steps 2 and 3 until the stack is empty.



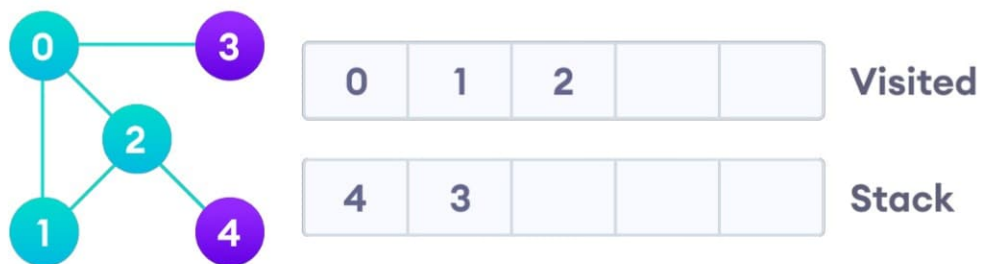
We start from vertex 0, the DFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the stack.



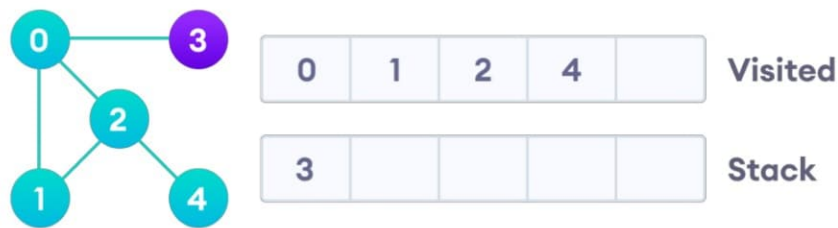
Next, we visit the element at the top of stack i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.



Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.

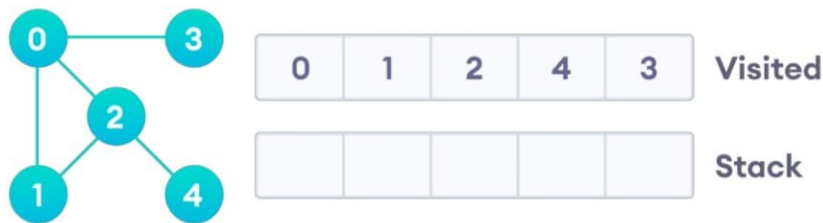


Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.



Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.

After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.



After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.

Applications of Depth First Search

Depth-first search (DFS) is an algorithm (or technique) for traversing a graph. Following are the problems that use DFS as a building block.

1) For an unweighted graph, DFS traversal of the graph produces the minimum spanning tree and all pair shortest path tree.

2) Detecting cycle in a graph

A graph has cycle if and only if we see a back edge during DFS. So we can run DFS for the graph and check for back edges. (See this for details)

3) Path Finding

We can specialize the DFS algorithm to find a path between two given vertices u and z .

- Call $\text{DFS}(G, u)$ with u as the start vertex.
- Use a stack S to keep track of the path between the start vertex and the current vertex.
- As soon as destination vertex z is encountered, return the path as the contents of the stack

4) Topological Sorting

5) To test if a graph is bipartite

We can augment either BFS or DFS when we first discover a new vertex, color it opposite its parents, and for each other edge, check it doesn't link two vertices of the same color. The first vertex in any connected component can be red or black! See this for details.

6) Finding Strongly Connected Components of a graph A directed graph is called strongly connected if there is a path from each vertex in the graph to every other vertex. (See this for DFS based also for finding Strongly Connected Components)

C program to implement the Graph Traversal

(a) Breadth first traversal

(b) Depth first traversal

DFS search starts from root node then traversal into left child node and continues, if item found it stops otherwise it continues. The advantage of DFS is it requires less memory compare to Breadth First Search (BFS).

BFS search starts from root node then traversal into next level of graph or tree and continues, if item found it stops otherwise it continues. The disadvantage of BFS is it requires more memory compare to Depth First Search (DFS).

Minimum Spanning Tree

Kruskal's Algorithm

Kruskal's algorithm is a minimum spanning tree algorithm that takes a graph as input and finds the subset of the edges of that graph which

- form a tree that includes every vertex
- has the minimum sum of weights among all the trees that can be formed from the graph

We start from the edges with the lowest weight and keep adding edges until we reach our goal.

The steps for implementing Kruskal's algorithm are as follows:

1. Sort all the edges from low weight to high
2. Take the edge with the lowest weight and add it to the spanning tree. If adding the edge created a cycle, then reject this edge.
3. Keep adding edges until we reach all vertices.

Algorithm:

KRUSKAL(G):

$A = \emptyset$

For each vertex $v \in G.V$:

MAKE-SET(v)

For each edge $(u, v) \in G.E$ ordered by increasing order by $\text{weight}(u, v)$:

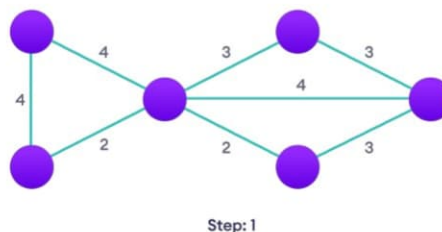
if $\text{FIND-SET}(u) \neq \text{FIND-SET}(v)$:

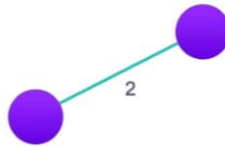
$A = A \cup \{(u, v)\}$

UNION(u, v)

return A

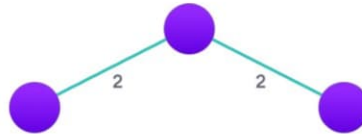
Example





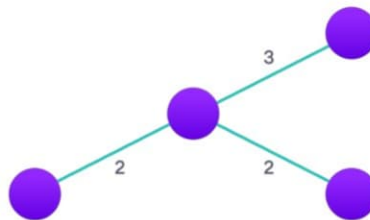
Step: 2

Choose the edge with the least weight, if there are more than 1, choose anyone



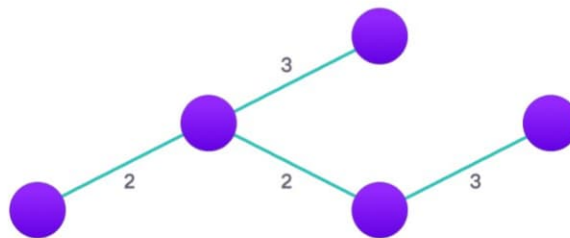
Step: 3

Choose the next shortest edge and add it



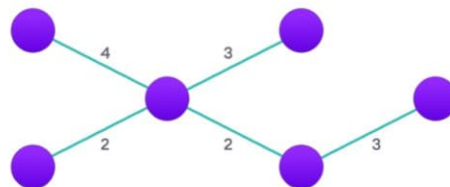
Step: 4

Choose the next shortest edge that doesn't create a cycle and add it



Step: 5

Choose the next shortest edge that doesn't create a cycle and add it



Step: 6

Repeat until you have a spanning tree

Prims Algorithm

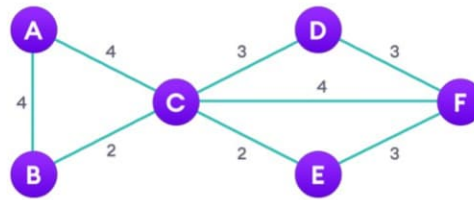
Prim's algorithm is a minimum spanning tree algorithm that takes a graph as input and finds the subset of the edges of that graph which

- form a tree that includes every vertex
- has the minimum sum of weights among all the trees that can be formed from the graph.

Pseudocode

The steps for implementing Prim's algorithm are as follows:

1. Initialize the minimum spanning tree with a vertex chosen at random.
2. Find all the edges that connect the tree to new vertices, find the minimum and add it to the tree
3. Keep repeating step 2 until we get a minimum spanning tree



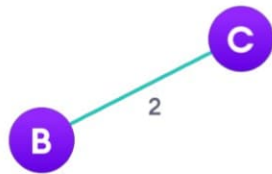
Step: 1

Start with a weighted graph



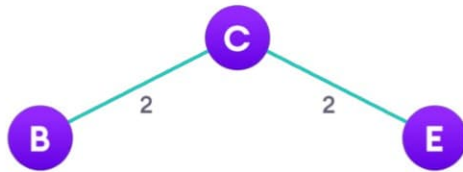
Step: 2

Choose a vertex



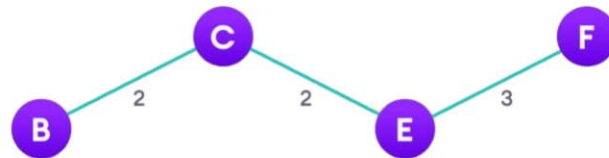
Step: 3

Choose the shortest edge from this vertex and add it



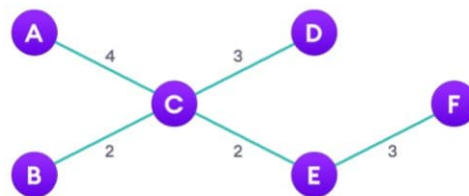
Step: 4

Choose the nearest vertex not yet in the solution



Step: 5

Choose the nearest edge not yet in the solution, if there are multiple choices, choose one at random



Step: 6

Repeat until you have a spanning tree

Shortest Path Algorithm

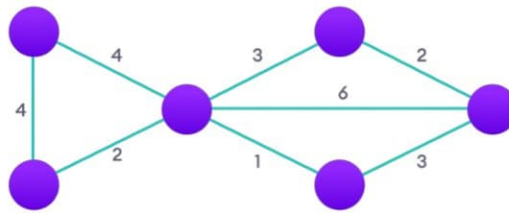
The shortest path problem is about finding a path between 2 vertices in a graph such that the total sum of the edges weights is minimum.

Dijkstra Algorithm

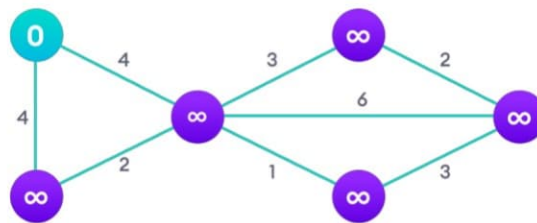
Dijkstra algorithm is a single-source shortest path algorithm. Here, single-source means that only one source is given, and we have to find the shortest path from the source to all the nodes.

Algorithm Steps:

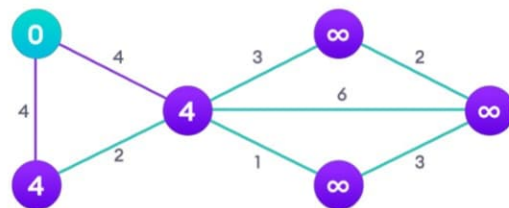
- Set all vertices distances = infinity except for the source vertex, set the source distance = 0.
- Push the source vertex in a min-priority queue in the form (distance, vertex), as the comparison in the min-priority queue will be according to vertices distances.
- Pop the vertex with the minimum distance from the priority queue (at first the popped vertex = source).
- Update the distances of the connected vertices to the popped vertex in case of "current vertex distance + edge weight < next vertex distance", then push the vertex with the new distance to the priority queue.
- If the popped vertex is visited before, just continue without using it.
- Apply the same algorithm again until the priority queue is empty.



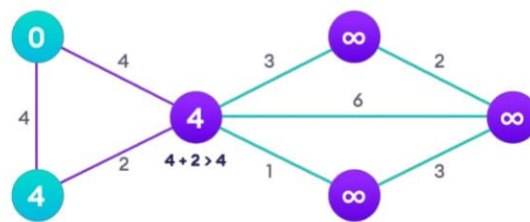
Start with a weighted graph



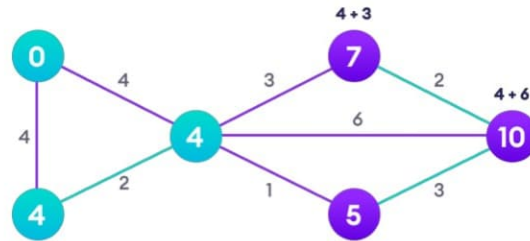
Choose a starting vertex and assign infinity path values to all other devices



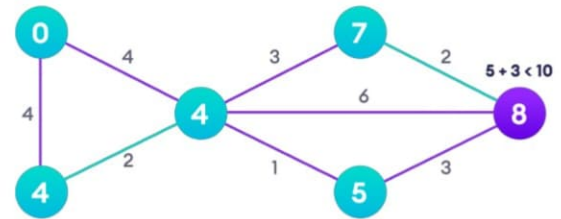
Go to each vertex and update its path length



If the path length of the adjacent vertex is lesser than new path length, don't update it



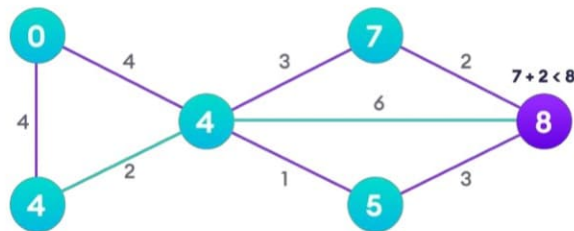
Step: 5



Step: 6

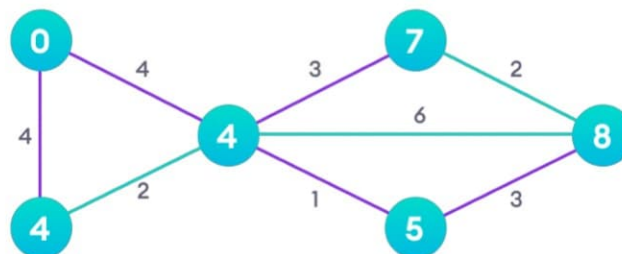
Avoid updating path lengths of already visited vertices

After each iteration, we pick the unvisited vertex with the least path length. So we choose 5 before 7



Step: 7

Notice how the rightmost vertex has its path length updated twice



Step: 8

Repeat until all the vertices have been visited

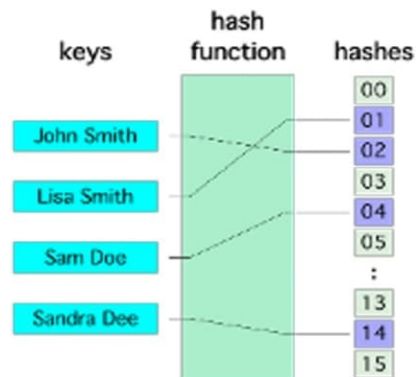
Hashing and Collision:

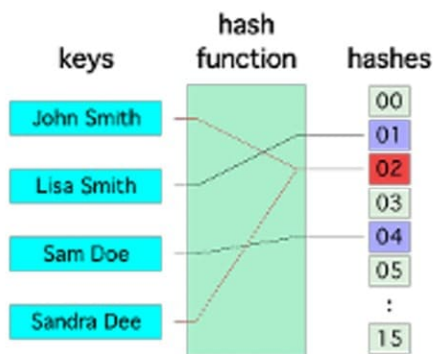
Hashing is the technique used for performing almost constant time search in case of insertion, deletion and find operation. Taking a very simple example of it, an array with its index as key is the example of hash table. So each index (key) can be used for accessing the value in a constant search time. This mapping key must be simple to compute and must help in identifying the associated value. Function which helps us in generating such kind of key-value mapping is known as Hash Function.

In a hashing system the keys are stored in an array which is called the Hash Table. A perfectly implemented hash table would always promise an average insert/delete/retrieval time of $O(1)$.

Hashing Function:

A function which employs some algorithm to compute the key K for all the data elements in the set U , such that the key K which is of a fixed size. The same key K can be used to map data to a hash table and all the operations like insertion, deletion and searching should be possible. The values returned by a **hash function** are also referred to as **hash** values, **hash** codes, **hash** sums, or **hashes**.





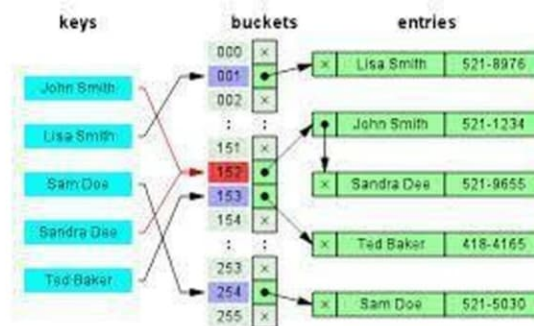
Hash Collision:

A situation when the resultant hashes for two or more data elements in the data set U , maps to the same location in the hash table, is called a hash collision. In such a situation two or more data elements would qualify to be stored / mapped to the same location in the hash table.

Hash collision resolution techniques:

Open Hashing (Separate chaining):

Open Hashing, is a technique in which the data is not directly stored at the hash key index (k) of the Hash table. Rather the data at the key index (k) in the hash table is a pointer to the head of the data structure where the data is actually stored. In the most simple and common implementations the data structure adopted for storing the element is a linked-list.



In this technique when a data needs to be searched, it might become necessary (worst case) to traverse all the nodes in the linked list to retrieve the data.

Note that the order in which the data is stored in each of these linked lists (or other data structures) is completely based on implementation requirements. Some of the popular criteria are insertion order, frequency of access etc.

Closed hashing (open Addressing)

In this technique a hash table with pre-identified size is considered. All items are stored in the hash table itself. In addition to the data, each hash bucket also maintains the three states: EMPTY, OCCUPIED, DELETED. While inserting, if a collision occurs, alternative cells are tried until an empty bucket is found. For which one of the following technique is adopted.

- 1. Liner Probing
- 2. Quadratic probing
- 3. Double hashing (in short in case of collision another hashing function is used with the key value as an input to identify where in the open addressing scheme the data should actually be stored.)

A comparative analysis of Closed Hashing vs Open Hashing

Open Addressing	Closed Addressing
All elements would be stored in the Hash table itself. No additional data structure is needed.	Additional Data structure needs to be used to accommodate collision data.
In cases of collisions, a unique hash key must be obtained.	Simple and effective approach to collision resolution. Key may or may not be unique.
Determining size of the hash table, adequate enough for storing all the data is difficult.	Performance deterioration of closed addressing much slower as compared to Open addressing.
State needs be maintained for the data (additional work)	No state data needs to be maintained (easier to maintain)
Uses space efficiently	Expensive on space

Applications of Hashing:

A **hash function** maps a variable length input string to fixed length output string -- its **hash value**, or **hash** for short. If the input is longer than the output, then some inputs must map to the same output -- a **hash collision**. Comparing the hash values for two inputs can give us one of two answers: the inputs are definitely not the same, or there is a possibility that they are the same. Hashing as we know it is used for performance improvement, error checking, and authentication. One example of a performance improvement is the common hash table, which uses a hash function to index into the correct bucket in the hash table, followed by comparing each element in the bucket to find a match. In error checking, hashes (checksums, message digests, etc.) are used to detect errors caused by either hardware or software. Examples are TCP checksums, ECC memory, and MD5 checksums on downloaded files. In this case, the hash provides additional assurance that the data we received is correct. Finally, hashes are used to authenticate messages. In this case, we are trying to protect the original input from tampering, and we select a hash that is strong enough to make malicious attack infeasible or unprofitable.

- ☐ Construct a *message authentication code* (MAC)
- ☐ Digital signature
- ☐ Make commitments, but reveal message later
- ☐ Timestamping
- ☐ Key updating: key is hashed at specific intervals resulting in new key